COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	07/016/2026	**DATE SUBMITTED:**	07/16//2026
NAME:	Lance Cristopher S. Salcedo	**INSTRUCTOR:**	Engr. Jimlord Quejardo

HANDS ON ACTIVITY NO. 2.1	**ARRAYS, POINTERS AND DYNAMIC MEMORY ALLOCATION**

1. PROCEDURE OUTPUT

COMPUTER ENGINEERING

QUEZON CITY

ILO A:

Screenshot

Code:

```cpp
#include <iostream>
#include <string.h>
class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName = "Lance Salcedo", int newAge = 67){
            studentName = newName;
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };
        //deconstructor
        ~Student(){
            std::cout << "Deconstructor Called." << std::endl;
        }
        //copy constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called." << std::endl;
            studentName = copyStudent.studentName ;
            studentAge = copyStudent.studentAge;
        }
        //display attributes
        void printDetails(){
            std::cout << this->studentName << "" << this->studentAge << std::endl;
        }
    };
int main(){
    Student student1("Joma", 19);
    Student student2(student1); //copy constructor
    Student student3; //copy assignment operator
    student3 = student2;

    return 0;
}
```

Output:

```
Constructor Called.
Copy Constructor Called.
Constructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.

--------------------------------
Process exited after 0.0127 seconds with return value 0
Press any key to continue . . .
```

Observation	I observed that the code that was given was used two header files which are #include <iostream> and #include <string.h>. The header file #include <string.h> is used in this code, so it can be manipulated. In the class students, we used private and public. In the private, this is where we store both the student's name and the age and also this means that it can't be accessed directly in the public. In the public, the code was used a constructor to create an object declaration, a destructor so it automatically destroys the object and copy constructor, so it will copy the constructor itself each has their print code if it was called. In the void function, we used it to print the object's name and age. Back to the main function, we used an object so it could print the object's constructor which is Student student1("Joma", 19);, then in the second object which is Student student2(student1) it copies the constructor and at student student3 it copies the assignment operator, before the end of the program each object is deleted using the deconstructor which also prints the deconstructor and returns to 0.

Table 2-1. Initial Driver Program

Screenshot

Code:

```cpp
#include <iostream>
#include <string.h>
class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName = "Lance Salcedo", int newAge = 67){
            studentName = newName;
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };
        //deconstructor
        ~Student(){
            std::cout << "Deconstructor Called." << std::endl;
        }
        //copy constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called." << std::endl;
            studentName = copyStudent.studentName ;
            studentAge = copyStudent.studentAge;
        }
        //display attributes
        void printDetails(){
            std::cout << this->studentName << "" << this->studentAge << std::endl;
        }
    };
int main(){
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

Output:

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.
Deconstructor Called.

--------------------------------
Process exited after 0.01267 seconds with return value 0
Press any key to continue . . .
```

Observation	Asides from what I observed on table 2.1, the main function now has a new code to run. The main function has a code where it has a constant size (or const size in the code), which means that it can't be changed so it always be 5. The object also now creates 5 arrays, each object is created in the constructor which means it will be called 5 times and it will be deleted in the deconstructor 5 more times before return it 0.

Table 2-2. Modified Driver Program with Student Lists

ILO B:

Loop A

Code:

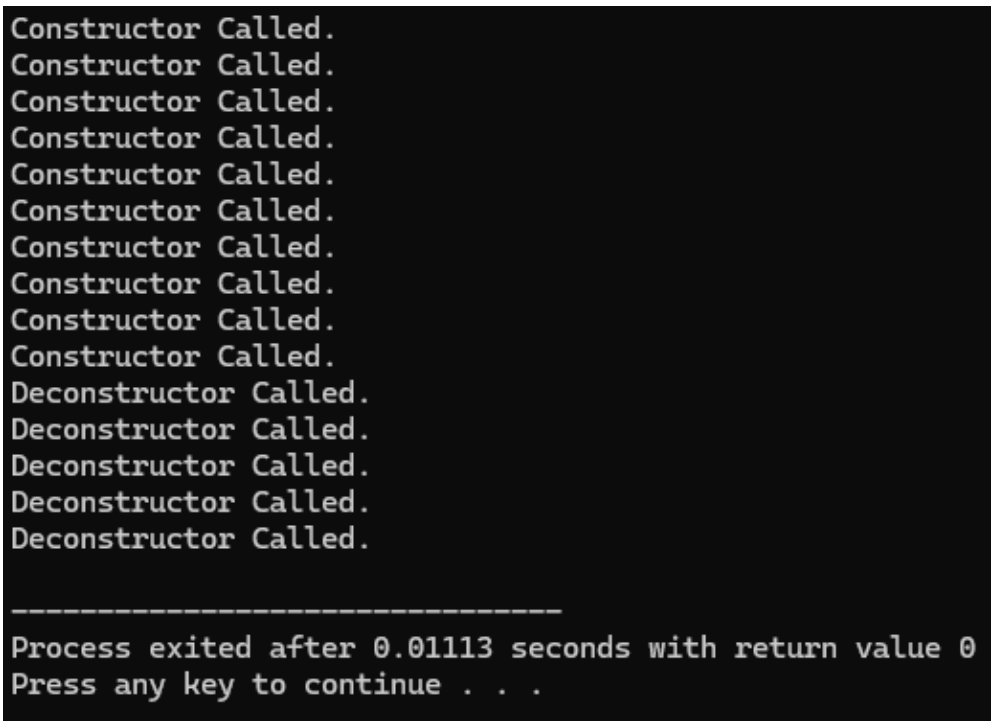
```cpp
#include <iostream>
#include <string.h>
class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName = "Lance Salcedo", int newAge = 67){
            studentName = newName;
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };
        //deconstructor
        ~Student(){
            std::cout << "Deconstructor Called." << std::endl;
        }
        //copy constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called." << std::endl;
            studentName = copyStudent.studentName ;
            studentAge = copyStudent.studentAge;
        }
        //display attributes
        void printDetails(){
            std::cout << this->studentName << "" << this->studentAge << std::endl;
        }
    };
int main(){
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    for(int i = 0; i < j; i++){
        Student *ptr = new Student(namesList[i], ageList[i]); //Loop A
        studentList [i] = *ptr;
    }

    return 0;
}
```

Output:

	
Observation	Asides the main function in table 2.2, we added another code that was given which is loop A. In loop A, the code has an increment. It also creates a new object on the heap which means it has dynamic memory and also has a pointer. The *ptr dereferences the object which copies the data of the element of the array (or studentList) which copies the assignment operator. Focusing on the main function, I observed that the constant size is 5 just like from the previous table. The code used j as the variable and once it to 5 it becomes false and it won't increment. The next line of text in the code now creates the array but since it wasn't given any names to use it proceeds to the constructor which happened 5 times. The next line proceeds to create the array in the strings list which won't be used until the for loop that was given in the code (Loop A).
Loop B	Code:

```cpp
#include <iostream>
#include <string.h>
class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName = "Lance Salcedo ", int newAge = 19){
            studentName = newName;
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };
        //deconstructor
        ~Student(){
            std::cout << "Deconstructor Called." << std::endl;
        }
        //copy constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called." << std::endl;
            studentName = copyStudent.studentName ;
            studentAge = copyStudent.studentAge;
        }
        //display attributes
        void printDetails(){
            std::cout << this->studentName << "" << this->studentAge << std::endl;
        }
    };
int main(){
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    for(int i = 0; i < j; i++){
        studentList[i].printDetails(); //loop B
    }

    return 0;
}
```

Output:

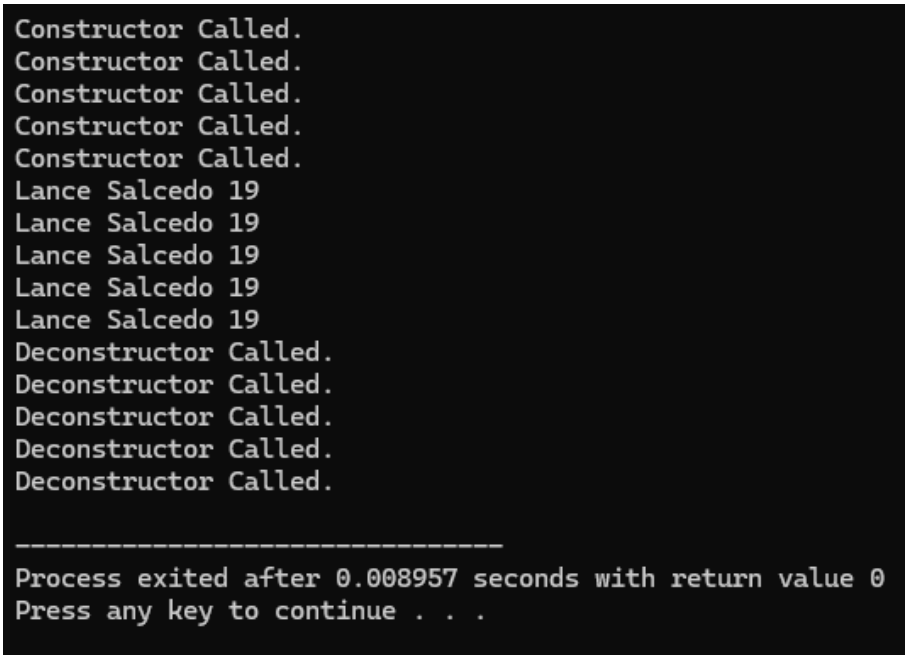
	
Observation	Besides from the previous observations in the previous tables, let's focus on the main function and the code that was given in the Loop B. After the for loop called the default constructor 5 times, it proceeds now to call the deconstructor 5 times before it return to 0.

Table 2-3. Final Driver Program

Modifications

Code:

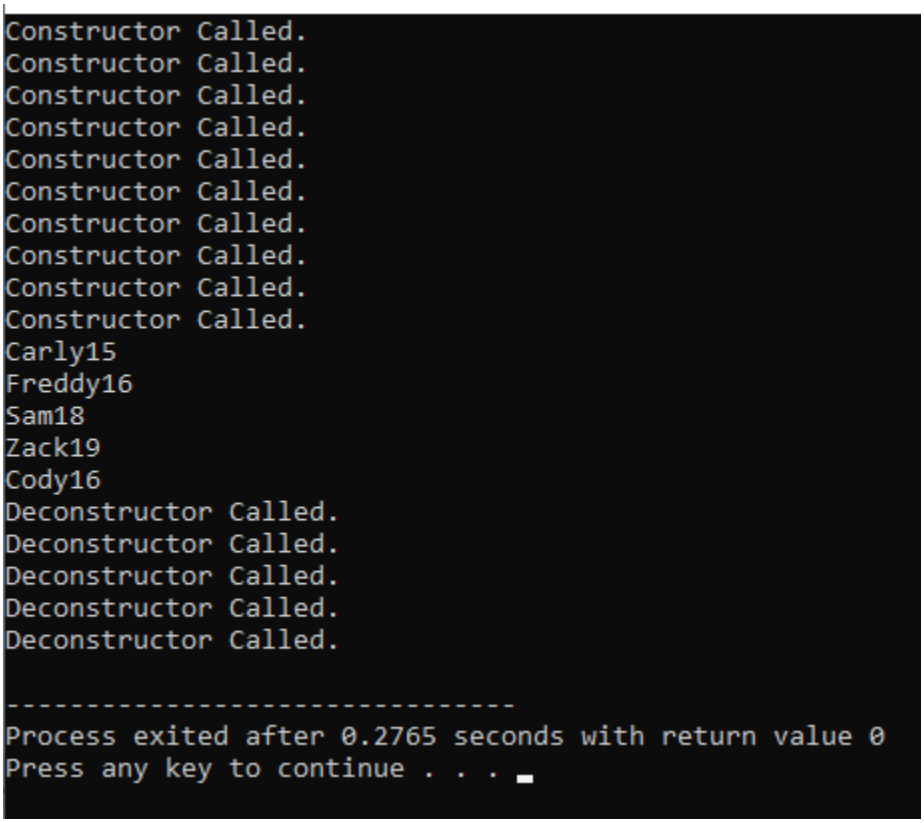
```cpp
#include <iostream>
#include <string.h>
class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName = "Lance Salcedo", int newAge = 19){
            studentName = newName;
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        };
        //deconstructor
        ~Student(){
            std::cout << "Deconstructor Called." << std::endl;
        }
        //copy constructor
        Student(const Student &copyStudent){
            std::cout << "Copy Constructor Called." << std::endl;
            studentName = copyStudent.studentName ;
            studentAge = copyStudent.studentAge;
        }

        //display attributes
        void printDetails(){
            std::cout << this->studentName << "" << this->studentAge << std::endl;
        }
    };
int main(){
    const size_t j = 5;

    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    for(int i = 0; i < j; i++){
        Student *ptr = new Student(namesList[i], ageList[i]); //Loop A
        studentList [i] = *ptr;
    }
    for(int i = 0; i < j; i++){
        studentList[i].printDetails(); //Loop B
    }
    Student* ptr;
    ptr = NULL;
    return 0;
}
```

Output:



Observation	Besides the previous observations from all of the previous tables, I am going to focus on the main function of the code. The modifications that I did are combining both loop A and loop B to see what will happen and the addition of nullptr. The result is that the constructor was called 10 times. This happens because after we create the previous studentList, it creates another list, this time its a new list. However, since the deconstructor was only called 5 times since the array has 5 characters, the other 5 were not called for a deconstructor. This actually creates a memory leak in the dynamic allocation (heap).

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

To be followed.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

COMPUTER ENGINEERING

QUEZON CITY

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used ChatGPT to generate ideas that I'll use on what I observed in the given codes.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.